Bikkad IT

Dream Big! Start Small!!! Act Now!!!

**Bikkad IT Institute**

# Session 1: Python Basics

*Practice Questions with Python Code Answers*

*Compiled by Bikkad IT Institute | Teaching Notes*

## WHAT IS INSIDE


1. Python Outputs
2. Python Data Types
3. Variables
4. Comments in Python
5. Keywords & Identifiers
6. User Input
7. Type Conversion
8. Literals


*48 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced*

BEGINNER INTERMEDIATE ADVANCED


OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

# Session 1: Python Basics - Practice Solutions

**Prepared for:** Bikkad IT Institute Students
**Topics:** Python Outputs, Python Data Types, Variables, Comments in Python, Keywords & Identifiers, User Input, Type Conversion, Literals
**Levels:** 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
**Date:** October 2026

## 1. Python Outputs

### Q1. Shop Name Board

BEGINNER

A grocery store wants its name board printed on the billing screen. Print the store name "Shree Ganesh Kirana Stores", the address "Station Road, Nashik" and the phone number "0253-2456789" on three separate lines.

**Solution:**

```
store_name = "Shree Ganesh Kirana Stores"
address = "Station Road, Nashik"
phone = "0253-2456789"

print(store_name)
print(address)
print(phone)
```

**Output:**

```
Shree Ganesh Kirana Stores
Station Road, Nashik
0253-2456789
```

## Q2. Train Ticket in One Line

BEGINNER

Print the details of a train ticket in a single line using the sep parameter so that every value is separated by " | ". Details: PNR 4521378960, From Pune, To Mumbai, Coach S4, Seat 32.

**Solution:**

```
print(4521378960, "Pune", "Mumbai", "S4", 32, sep=" | ")
```

**Output:**

```
4521378960 | Pune | Mumbai | S4 | 32
```

## Q3. Date and Time Display

INTERMEDIATE

A digital clock app stores day, month and year as separate numbers (17, 10, 2026) and hours, minutes as (9, 45). Print the date in DD-MM-YYYY style and the time in HH:MM style using only the sep parameter.

**Solution:**

```
day = 17
month = 10
year = 2026
hours = 9
minutes = 45

print(day, month, year, sep="-")
print(hours, minutes, sep=":")
```

**Output:**

```
17-10-2026
9:45
```

## Q4. Download Progress Message

INTERMEDIATE

Show a file download status on ONE line using two separate print() statements. The first print should show "Downloading report.pdf" followed by "..." (use the end parameter) and the second print should show "Completed".

**Solution:**

```
print("Downloading report.pdf", end="...")
print("Completed")
```

**Output:**

```
Downloading report.pdf...Completed
```

## Q5. Restaurant Bill Receipt

ADVANCED

Print a neat restaurant receipt for "Hotel Sahyadri" with a line of 30 dashes above and below. Show 3 items (Misal Pav 80, Vada Pav 30, Masala Tea 20) as "item : price" using sep, and print the total on the last line. Calculate the total inside print().

**Solution:**

```
print("-" * 30)
print("Hotel Sahyadri")
print("-" * 30)
print("Misal Pav", 80, sep=" : ")
print("Vada Pav", 30, sep=" : ")
print("Masala Tea", 20, sep=" : ")
print("-" * 30)
print("Total", 80 + 30 + 20, sep=" : ")
print("-" * 30)
```

**Output:**

```
------------------------------
Hotel Sahyadri
------------------------------
Misal Pav : 80
Vada Pav : 30
Masala Tea : 20
------------------------------
Total : 130
------------------------------
```

### Q6. Export Rows Like a CSV File

ADVANCED

An HR system exports employee data to a CSV file. Print a header row (EmpID, Name, Dept, Salary) and two employee rows using sep=",". Then print all three department names on one line separated by " / " using the end parameter in three separate print() calls.

**Solution:**

```
print("EmpID", "Name", "Dept", "Salary", sep=",")
print(101, "Sneha Patil", "Sales", 32000, sep=",")
print(102, "Rohan Shinde", "IT", 45000, sep=",")

print("Sales", end=" / ")
print("IT", end=" / ")
print("HR")
```

**Output:**

```
EmpID,Name,Dept,Salary
101,Sneha Patil,Sales,32000
102,Rohan Shinde,IT,45000
Sales / IT / HR
```

## 2. Python Data Types

### Q7. Student Profile

BEGINNER

Store a student's roll number (21), percentage (78.5), name ("Kavya Joshi") and pass status (True) in four variables. Print the type of each variable.

**Solution:**

```
roll_no = 21
percentage = 78.5
name = "Kavya Joshi"
is_passed = True

print(type(roll_no))
print(type(percentage))
print(type(name))
print(type(is_passed))
```

**Output:**

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

## Q8. Online Shopping Data

BEGINNER

An online shopping app stores: a cart as a list ["Shoes", "Watch"], the warehouse GPS location as a tuple (18.52, 73.85), one product as a dictionary {"name": "Bag", "price": 899} and unique categories as a set {"Fashion", "Electronics"}. Create them and print the type of each.

**Solution:**

```
cart = ["Shoes", "Watch"]
location = (18.52, 73.85)
product = {"name": "Bag", "price": 899}
categories = {"Fashion", "Electronics"}

print(type(cart))
print(type(location))
print(type(product))
print(type(categories))
```

**Output:**

```
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
```

## Q9. Why Account Number is Text

INTERMEDIATE

A bank account number is "00451236". Store it once as a string and once as an integer and print both. Observe what happens to the leading zeros and decide which data type is correct for account numbers.

**Solution:**

```
account_str = "00451236"
account_int = 451236

print(account_str)
print(account_int)
print(type(account_str))
print(type(account_int))
# Leading zeros are lost in int,
# so account numbers must be stored as str
```

**Output:**

```
00451236
451236
<class 'str'>
<class 'int'>
```

## Q10. Unique Delivery Cities

INTERMEDIATE

A courier company received deliveries for the cities Pune, Nashik, Pune, Mumbai, Nashik, Pune. Store them in a set to keep only unique cities. Print the set and the number of unique cities.

**Solution:**

```
cities = {"Pune", "Nashik", "Pune", "Mumbai", "Nashik", "Pune"}

print(cities)
print(len(cities))
```

**Output:**

```
{'Pune', 'Mumbai', 'Nashik'}
3
```

## Q11. Employee Record with Nested Data

ADVANCED

Create a dictionary for an employee with keys: name ("Ajay Kale"), age (29), salary (55000.0), skills (a list ["Excel", "SQL", "Python"]) and is_permanent (True). Print the type of every value and the second skill from the list.

**Solution:**

```
employee = {
    "name": "Ajay Kale",
    "age": 29,
    "salary": 55000.0,
    "skills": ["Excel", "SQL", "Python"],
    "is_permanent": True
}

print(type(employee["name"]))
print(type(employee["age"]))
print(type(employee["salary"]))
print(type(employee["skills"]))
print(type(employee["is_permanent"]))
print(employee["skills"][1])
```

**Output:**

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'list'>
<class 'bool'>
SQL
```

### Q12. Validate Price Data Type

ADVANCED

Prices of three items came from a form as 450, "450" and 450.0. For each value, check whether its type is int, str or float using type() with the == operator and print the result (True/False).

**Solution:**

```
price1 = 450
price2 = "450"
price3 = 450.0

print(type(price1) == int)
print(type(price2) == str)
print(type(price3) == float)
print(type(price2) == int)
```

**Output:**

```
True
True
True
False
```

## 3. Variables

### Q13. Mobile Recharge Plan

BEGINNER

Store the details of a mobile recharge plan in variables: plan name "Super Saver", price 299, validity 28 days and data per day 1.5 GB. Print each variable.

**Solution:**

```
plan_name = "Super Saver"
price = 299
validity_days = 28
data_per_day_gb = 1.5

print(plan_name)
print(price)
print(validity_days)
print(data_per_day_gb)
```

**Output:**

```
Super Saver
299
28
1.5
```

## Q14. Class Teachers in One Line

BEGINNER

Assign the names of three class teachers (Maths: "Mr. Pawar", Science: "Mrs. Deshmukh", English: "Ms. Rao") to three variables in ONE line using multiple assignment and print them.

**Solution:**

```
maths, science, english = "Mr. Pawar", "Mrs. Deshmukh", "Ms. Rao"

print(maths)
print(science)
print(english)
```

**Output:**

```
Mr. Pawar
Mrs. Deshmukh
Ms. Rao
```

## Q15. Swap Shift Workers

INTERMEDIATE

In a factory, "Mahesh" works the morning shift and "Suresh" works the evening shift. Swap their shifts using Python's one-line swap and print both shifts before and after swapping.

**Solution:**

```
morning = "Mahesh"
evening = "Suresh"
print(morning)
print(evening)

morning, evening = evening, morning
print(morning)
print(evening)
```

**Output:**

```
Mahesh
Suresh
Suresh
Mahesh
```

## Q16. Wallet Balance Updates

INTERMEDIATE

A digital wallet starts with balance 1000. The user pays 250 for groceries and later adds 500 cash-back. Reassign the same variable after each transaction and print the balance after every step.

**Solution:**

```
balance = 1000
print(balance)

balance = balance - 250
print(balance)

balance = balance + 500
print(balance)
```

**Output:**

```
1000
750
1250
```

## Q17. Shopping Cart Final Amount

ADVANCED

A cart has three items priced 1200, 850 and 450. Store the subtotal in a variable, then a 10% discount, then 18% GST on the discounted amount, and finally the payable amount. Use a separate variable for each step and print all of them.

**Solution:**

```
item1 = 1200
item2 = 850
item3 = 450

subtotal = item1 + item2 + item3
print(subtotal)

discount = subtotal * 10 / 100
print(discount)

after_discount = subtotal - discount
print(after_discount)

gst = after_discount * 18 / 100
print(gst)

payable = after_discount + gst
print(payable)
```

**Output:**

```
2500
250.0
2250.0
405.0
2655.0
```

### Q18. Shared Counters with id()

ADVANCED

Create three counters for a parking lot (cars, bikes, trucks) all set to 0 in one line (x = y = z style). Print their id() to see they point to the same object. Then update cars to 5 and print the id() of cars and bikes again. Explain the change in a comment.

**Solution:**

```
cars = bikes = trucks = 0
print(id(cars) == id(bikes))
print(id(bikes) == id(trucks))

cars = 5
print(cars)
print(bikes)
print(id(cars) == id(bikes))
# cars now points to a new object (5); bikes still points to 0
```

**Output:**

```
True
True
5
0
False
```

## 4. Comments in Python

## Q19. Temperature Log with Comments

BEGINNER

Write a small program that stores the morning (24) and afternoon (33) temperature of Pune and calculates the difference. Add a single-line comment above each step explaining it.

**Solution:**

```
# Morning temperature in Celsius
morning_temp = 24

# Afternoon temperature in Celsius
afternoon_temp = 33

# Difference between afternoon and morning
difference = afternoon_temp - morning_temp

# Show the result
print(difference)
```

**Output:**

```
9
```

## Q20. Program Header Block

BEGINNER

Add a multi-line comment (triple quotes) at the top of an electricity bill program that mentions Program name, Purpose, Author and Date. Then calculate the bill for 120 units at Rs. 7 per unit.

**Solution:**

```
"""
Program : Electricity Bill Calculator
Purpose : Calculate monthly bill amount
Author  : Bikkad IT Student
Date    : October 2026
"""

units = 120
rate = 7
bill = units * rate
print(bill)
```

**Output:**

```
840
```

## Q21. Comment Out Debug Lines

INTERMEDIATE

In an EMI program (loan 120000, 12 months, no interest), there are two debug print lines. Comment them out so that only the final EMI is printed.

**Solution:**

```
loan = 120000
months = 12

# print(loan)      # debug line - disabled
# print(months)    # debug line - disabled

emi = loan / months
print(emi)
```

**Output:**

```
10000.0
```

## Q22. Inline Comments for GST

INTERMEDIATE

Calculate the final price of a laptop (base price 45000, GST 18%). Write inline comments (at the end of each line) to explain every line of code.

**Solution:**

```
base_price = 45000              # price before tax
gst_rate = 18                   # GST percentage
gst_amount = base_price * gst_rate / 100   # tax amount
final_price = base_price + gst_amount      # price customer pays
print(final_price)              # display final price
```

**Output:**

```
53100.0
```

## Q23. Comment vs String Value

ADVANCED

Show the difference between a real comment (#) and a triple-quoted string. Write a # comment, a triple-quoted string that is NOT assigned, and a triple-quoted string that IS assigned to a variable named note. Print note and explain in comments which ones Python ignores.

**Solution:**

```
# This is a comment - Python ignores it completely

'''
This string is not stored anywhere.
Python creates it and throws it away,
so it behaves like a comment.
'''

note = '''Delivery between
10 AM and 6 PM'''

print(note)
# Only 'note' is a real value; the other two produce no output
```

**Output:**

```
Delivery between
10 AM and 6 PM
```

### Q24. Explain WHY, Not WHAT

ADVANCED

A mall parking charges Rs. 20 per hour but the first 15 minutes are free. A car stayed 135 minutes. Calculate the parking fee. Write comments that explain WHY each step is done (business rule), not just what the code does.

**Solution:**

```
total_minutes = 135

# First 15 minutes are free as per mall policy,
# so they are not counted in the bill
free_minutes = 15
billable_minutes = total_minutes - free_minutes

# Mall charges for every started hour, so round UP
# using -(-a // b) which is ceiling division
hours = -(-billable_minutes // 60)

rate_per_hour = 20
fee = hours * rate_per_hour
print(fee)
```

**Output:**

```
40
```

## 5. Keywords & Identifiers

### Q25. List All Keywords

BEGINNER

Import the keyword module, print the list of all Python keywords and print how many keywords your Python version has.

**Solution:**

```
import keyword

all_keywords = keyword.kwlist
print(all_keywords)

total = len(all_keywords)
print(total)
```

**Output:**

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async',
'await', 'break', 'class', 'continue', 'def', 'del', 'elif',
'else', 'except', 'finally', 'for', 'from', 'global', 'if',
'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
 'raise', 'return', 'try', 'while', 'with', 'yield']
35
```

## Q26. Is It a Keyword?

BEGINNER

Check whether the words "while", "price", "True" and "true" are Python keywords using keyword.iskeyword() and print the results.

**Solution:**

```
import keyword

print(keyword.iskeyword("while"))
print(keyword.iskeyword("price"))
print(keyword.iskeyword("True"))
print(keyword.iskeyword("true"))
```

**Output:**

```
True
False
True
False
```

## Q27. Fix the Hospital Variables

INTERMEDIATE

A hospital program has these invalid variable names: 1patient, patient-name, class, bed no, @ward. Rewrite the program with valid identifiers (patient_1, patient_name, class_name, bed_no, ward) and store suitable values. Print all values.

**Solution:**

```
patient_1 = "P1001"
patient_name = "Ramesh Gaikwad"
class_name = "General"
bed_no = 14
ward = "B-Wing"

print(patient_1)
print(patient_name)
print(class_name)
print(bed_no)
print(ward)
```

**Output:**

```
P1001
Ramesh Gaikwad
General
14
B-Wing
```

## Q28. Case-Sensitive Sales Report

INTERMEDIATE

Create three different variables total, Total and TOTAL for daily, weekly and monthly sales (5000, 35000, 150000). Print all three to prove that identifiers are case-sensitive.

**Solution:**

```
total = 5000       # daily sales
Total = 35000      # weekly sales
TOTAL = 150000     # monthly sales

print(total)
print(Total)
print(TOTAL)
```

**Output:**

```
5000
35000
150000
```

## Q29. Keyword or Built-in Function?

ADVANCED

Check whether "None", "print", "len" and "import" are keywords. You will find print and len are NOT keywords (they are built-in functions). Write a comment explaining why you still should not use them as variable names.

**Solution:**

```
import keyword

print(keyword.iskeyword("None"))
print(keyword.iskeyword("print"))
print(keyword.iskeyword("len"))
print(keyword.iskeyword("import"))

# print and len are built-in functions, not keywords.
# If we write  print = 10  Python allows it, but after that
# print() stops working in the program. So avoid such names.
```

**Output:**

```
True
False
False
True
```

### Q30. Naming Conventions in a Billing App

ADVANCED

Write a billing program following Python naming conventions: a CONSTANT for GST rate (UPPER_CASE), snake_case variables for item price and quantity, and an _internal variable for a temporary value. Calculate and print the bill with GST for 3 items priced 250 each.

**Solution:**

```
GST_RATE = 0.18          # constant - UPPER_CASE

item_price = 250         # snake_case variable
quantity = 3

_temp_amount = item_price * quantity   # temporary value
print(_temp_amount)

bill_amount = _temp_amount + _temp_amount * GST_RATE
print(bill_amount)
```

**Output:**

```
750
885.0
```

## 6. User Input

### Q31. Customer Greeting

BEGINNER

Ask the customer for their name and city using input() and print a welcome message with both values.

**Solution:**

```
name = input("Enter your name: ")
city = input("Enter your city: ")

print("Welcome", name, "from", city)
```

**Output (with sample input):**

```
Enter your name: Pooja
Enter your city: Aurangabad
Welcome Pooja from Aurangabad
```

## Q32. Age After 5 Years

BEGINNER

Take the user's age as input, convert it to an integer and print the age after 5 years.

**Solution:**

```
age = int(input("Enter your age: "))

future_age = age + 5
print(future_age)
```

**Output (with sample input):**

```
Enter your age: 19
24
```

## Q33. Movie Ticket Booking

INTERMEDIATE

Ask the user for the number of tickets and price per ticket (price can be decimal). Calculate and print the total amount.

**Solution:**

```
tickets = int(input("Number of tickets: "))
price = float(input("Price per ticket: "))

total = tickets * price
print(total)
```

**Output (with sample input):**

```
Number of tickets: 4
Price per ticket: 180.50
722.0
```

## Q34. Marks and Percentage

INTERMEDIATE

Take marks of three subjects (out of 100) from the user. Print the total marks and the percentage.

**Solution:**

```
maths = int(input("Maths marks: "))
science = int(input("Science marks: "))
english = int(input("English marks: "))

total = maths + science + english
print(total)

percentage = total / 300 * 100
print(percentage)
```

**Output (with sample input):**

```
Maths marks: 78
Science marks: 85
English marks: 69
232
77.33333333333333
```

## Q35. Petrol Cost for a Trip

ADVANCED

A family is going from Pune to Kolhapur. Take distance (km), car mileage (km per litre) and petrol price per litre as input. Calculate litres needed and total petrol cost, both rounded to 2 decimal places.

**Solution:**

```
distance = float(input("Distance in km: "))
mileage = float(input("Mileage (km/litre): "))
petrol_price = float(input("Petrol price per litre: "))

litres = distance / mileage
print(round(litres, 2))

cost = litres * petrol_price
print(round(cost, 2))
```

### Output (with sample input):

```
Distance in km: 230
Mileage (km/litre): 16
Petrol price per litre: 106.5
14.38
1530.94
```

## Q36. Simple Interest Calculator

ADVANCED

Take principal, annual interest rate and time in years as input. Calculate simple interest (SI = P x R x T / 100) and the total amount to be paid. Round both to 2 decimals.

### Solution:

```python
principal = float(input("Principal amount: "))
rate = float(input("Rate of interest (%): "))
years = float(input("Time in years: "))

si = principal * rate * years / 100
print(round(si, 2))

total_amount = principal + si
print(round(total_amount, 2))
```

### Output (with sample input):

```
Principal amount: 75000
Rate of interest (%): 8.5
Time in years: 3
19125.0
94125.0
```

# 7. Type Conversion

## Q37. Price from Text to Number

BEGINNER

A product price comes from a website as the string "499". Convert it to an integer and add a delivery charge of 40. Print the final price and its type.

**Solution:**

```
price_text = "499"
price = int(price_text)

final_price = price + 40
print(final_price)
print(type(final_price))
```

**Output:**

```
539
<class 'int'>
```

## Q38. Implicit Conversion in a Bill

BEGINNER

Add the number of items 3 (int) to a service charge 2.5 (float). Print the result and its type to see Python's automatic (implicit) conversion.

**Solution:**

```
items = 3
service_charge = 2.5

result = items + service_charge
print(result)
print(type(result))
```

**Output:**

```
5.5
<class 'float'>
```

## Q39. Marks Message with Concatenation

INTERMEDIATE

A student scored 87 marks. Create the message "Your score is 87" using the + operator. You must convert the number using str() first.

**Solution:**

```
marks = 87
message = "Your score is " + str(marks)
print(message)
```

**Output:**

```
Your score is 87
```

## Q40. Is the Cart Empty?

INTERMEDIATE

Use bool() to check: cart quantity 0, cart quantity 3, coupon code "" (empty) and coupon code "DIWALI50". Print the boolean value of each.

**Solution:**

```
print(bool(0))
print(bool(3))
print(bool(""))
print(bool("DIWALI50"))
```

**Output:**

```
False
True
False
True
```

## Q41. Truncate vs Round

ADVANCED

Fuel price is received as the string "106.75". Convert it to float, then show the difference between int() (which cuts the decimal) and round() (which rounds to the nearest whole number).

**Solution:**

```
price_text = "106.75"
price = float(price_text)
print(price)

cut_value = int(price)
print(cut_value)

rounded_value = round(price)
print(rounded_value)
```

**Output:**

```
106.75
106
107
```

## Q42. Convert Between Collections and Bases

ADVANCED

Order IDs [101, 102, 101, 103, 102] contain duplicates. Convert the list to a set to remove duplicates, then back to a list, and then to a tuple. Also convert the binary string "1010" and hexadecimal string "FF" to integers using int() with a base.

**Solution:**

```
order_ids = [101, 102, 101, 103, 102]

unique_ids = set(order_ids)
print(unique_ids)

id_list = list(unique_ids)
print(id_list)

id_tuple = tuple(id_list)
print(id_tuple)

binary_value = int("1010", 2)
print(binary_value)

hex_value = int("FF", 16)
print(hex_value)
```

**Output:**

```
{101, 102, 103}
[101, 102, 103]
(101, 102, 103)
10
255
```

# 8. Literals

## Q43. Numeric Literals in Inventory

BEGINNER

Store inventory data using numeric literals: quantity 150 (integer), weight 2.75 kg (float) and an electrical impedance value 3+4j (complex). Print each value and its type.

**Solution:**

```
quantity = 150
weight = 2.75
impedance = 3 + 4j

print(quantity)
print(type(quantity))
print(weight)
print(type(weight))
print(impedance)
print(type(impedance))
```

**Output:**

```
150
<class 'int'>
2.75
<class 'float'>
(3+4j)
<class 'complex'>
```

## Q44. Quotes Inside Strings

BEGINNER

Store the sentences It's a sunny day and Teacher said "Well done" in two string variables. Use double quotes for the first and single quotes for the second. Print both.

**Solution:**

```
sentence1 = "It's a sunny day"
sentence2 = 'Teacher said "Well done"'

print(sentence1)
print(sentence2)
```

**Output:**

```
It's a sunny day
Teacher said "Well done"
```

## Q45. Mini Invoice with Escape Sequences

INTERMEDIATE

Print a mini invoice using a single string with \n (new line) and \t (tab) so that the output shows Item and Price as columns for Pen 10 and Notebook 45.

**Solution:**

```
invoice = "Item\tPrice\nPen\t10\nNotebook\t45"
print(invoice)
```

**Output:**

```
Item    Price
Pen     10
Notebook        45
```

## Q46. Binary, Octal and Hex Literals

INTERMEDIATE

In computers, file permissions are written in octal (0o755), a status flag in binary (0b1101) and a colour value in hexadecimal (0xFF). Store these literals and print their decimal values.

**Solution:**

```
permission = 0o755
status_flag = 0b1101
red_value = 0xFF

print(permission)
print(status_flag)
print(red_value)
```

**Output:**

```
493
13
255
```

## Q47. Scientific Notation

ADVANCED

India's population is about 1.4e9 and a bacteria's size is 2.5e-6 metres. Store these using scientific notation. Print them, and calculate how many bacteria lined up end-to-end make 1 metre.

**Solution:**

```
population = 1.4e9
bacteria_size = 2.5e-6

print(population)
print(bacteria_size)

bacteria_in_one_metre = 1 / bacteria_size
print(round(bacteria_in_one_metre))
```

**Output:**

```
1400000000.0
2.5e-06
400000
```

## Q48. Special and Collection Literals

ADVANCED

A courier order is created but its delivery date is not decided yet - store it as None. Store the order status as True. Also create a menu using list, tuple, dictionary and set literals. Print all values and show that True + True gives 2.

**Solution:**

```
delivery_date = None
is_confirmed = True

menu_list = ["Poha", "Upma", "Idli"]
sizes = ("Small", "Medium", "Large")
prices = {"Poha": 30, "Upma": 35, "Idli": 40}
tags = {"veg", "breakfast"}

print(delivery_date)
print(type(delivery_date))
print(is_confirmed)
print(menu_list)
print(sizes)
print(prices)
print(tags == {"breakfast", "veg"})
print(True + True)
```

**Output:**

```
None
<class 'NoneType'>
True
['Poha', 'Upma', 'Idli']
('Small', 'Medium', 'Large')
{'Poha': 30, 'Upma': 35, 'Idli': 40}
True
2
```

### End of Solutions

*Prepared by: Bikkad IT Institute*
*For: Python Beginner Level Students*
*Date: October 2026*